

```
#=====
# INTRODUCTION
#=====
```

```
# This is meant to be a sample starter script if you choose to use Python
# for this case study. This is not comprehensive of everything you'll
# do in the case study, but should be used as a starting point if it is helpful for you.
```

```
#=====
INSTALL AND LOAD LIBRARIES
#=====
```

```
# You may need to install these libraries if you haven't already.
# You can do this by running the following commands in your terminal:
# pip install pandas
```

```
#Load relevant libraries
import pandas as pd
```

```
#=====
# LOAD CSV FILES
#=====
```

```
# Upload Divvy datasets (csv files)
q1_2019 = pd.read_csv("Divvy_Trips_2019_Q1.csv")
q1_2020 = pd.read_csv("Divvy_Trips_2020_Q1.csv")
```

```
#=====
# WRANGLE DATA AND COMBINE INTO A SINGLE FILE
#=====
```

```
# Compare column names for each of the files
print("Columns for Q1 2019:\n", q1_2019.columns)
print("\nColumns for Q1 2020:\n", q1_2020.columns)
```

```
# Rename columns to make them consistent with q1_2020
q1_2019 = q1_2019.rename(columns={
    'trip_id': 'ride_id',
    'bikeid': 'rideable_type',
    'start_time': 'started_at',
    'end_time': 'ended_at',
    'from_station_name': 'start_station_name',
    'from_station_id': 'start_station_id',
    'to_station_name': 'end_station_name',
```

```
    'to_station_id': 'end_station_id',  
    'usertype': 'member_casual'  
})
```

```
# Inspect the data frames and look for incongruities  
print("\nQ1 2019 Data Info:")  
q1_2019.info()  
print("\nQ1 2020 Data Info:")  
q1_2020.info()
```

```
# Convert ride_id and rideable_type to string so that they can stack correctly  
q1_2019['ride_id'] = q1_2019['ride_id'].astype(str)  
q1_2019['rideable_type'] = q1_2019['rideable_type'].astype(str)
```

```
# Stack individual quarter's data frames into one big data frame  
all_trips = pd.concat([q1_2019, q1_2020], ignore_index=True)
```

```
# Remove lat, long, birthyear, and gender fields as this data was dropped beginning in 2020  
# The 'errors="ignore"' argument prevents an error if a column is not found  
all_trips = all_trips.drop(columns=['start_lat', 'start_lng', 'end_lat', 'end_lng', 'birthyear', 'gender',  
    'tripduration'], errors='ignore')
```

```
#=====
```

```
# CLEAN UP AND ADD DATA TO PREPARE FOR ANALYSIS
```

```
#=====
```

```
# Inspect the new table that has been created  
print("\nList of column names:\n", all_trips.columns)  
print("\nHow many rows are in data frame?", len(all_trips))  
print("\nDimensions of the data frame:", all_trips.shape)  
print("\nSee the first 6 rows of data frame:\n", all_trips.head())  
print("\nSee list of columns and data types:\n")  
all_trips.info()  
# Statistical summary of data (mainly for numerics)  
print("\nStatistical summary of data:\n", all_trips.describe())
```

```
# There are a few problems you will need to fix:  
# (1) In the "member_casual" column, there are two names for members ("member" and  
    "Subscriber") and two names for casual riders ("Customer" and "casual"). You will need to  
    consolidate that from four to two labels.  
# (2) The data can only be aggregated at the ride-level, which is too granular. You will want to  
    add some additional columns of data -- such as day, month, year -- that provide additional  
    opportunities to aggregate the data.
```

(3) You will want to add a calculated field for length of ride since the 2020Q1 data did not have the "tripduration" column. We will add "ride_length" to the entire data frame for consistency.

(4) There are some rides where tripduration shows up as negative, including several hundred rides where Divvy took bikes out of circulation for Quality Control reasons. You will want to delete these rides.

```
# In the "member_casual" column, replace "Subscriber" with "member" and "Customer" with "casual"
```

```
# Begin by discovering how many observations fall under each usertype
```

```
print("\nValue counts for 'member_casual' column before cleaning:\n",
```

```
all_trips['member_casual'].value_counts())
```

```
# Reassign to the desired values (you can go with the current 2020 labels)
```

```
all_trips['member_casual'] = all_trips['member_casual'].replace({
```

```
    'Subscriber': 'member',
```

```
    'Customer': 'casual'
```

```
})
```

```
# Check to make sure the proper number of observations were reassigned
```

```
print("\nValue counts for 'member_casual' column after cleaning:\n",
```

```
all_trips['member_casual'].value_counts())
```

```
# Convert 'started_at' and 'ended_at' to datetime objects
```

```
all_trips['started_at'] = pd.to_datetime(all_trips['started_at'])
```

```
all_trips['ended_at'] = pd.to_datetime(all_trips['ended_at'])
```

```
# Add columns that list the date, month, day, and year of each ride
```

```
all_trips['date'] = all_trips['started_at'].dt.date
```

```
all_trips['month'] = all_trips['started_at'].dt.month
```

```
all_trips['day'] = all_trips['started_at'].dt.day
```

```
all_trips['year'] = all_trips['started_at'].dt.year
```

```
all_trips['day_of_week'] = all_trips['started_at'].dt.day_name()
```

```
# Add a "ride_length" calculation to all_trips (in seconds)
```

```
all_trips['ride_length'] = (all_trips['ended_at'] - all_trips['started_at']).dt.total_seconds()
```

```
# Inspect the structure of the columns
```

```
print("\nData types after adding date columns and ride_length:\n")
```

```
all_trips.info()
```

```
# Remove "bad" data
```

```
# The data frame includes entries where bikes were taken out of docks for quality checks or  
ride_length was negative
```

```
# Create a new version of the data frame (v2) since data is being removed
```

```

all_trips_v2 = all_trips[(all_trips['start_station_name'] != "HQ QR") & (all_trips['ride_length'] >=
0)].copy()

#=====
# CONDUCT DESCRIPTIVE ANALYSIS
#=====

# Descriptive analysis on ride_length (all figures in seconds)
print("\nDescriptive statistics for ride_length:\n", all_trips_v2['ride_length'].describe())

# Compare members and casual users
print("\nRide length statistics grouped by member_casual:\n",
all_trips_v2.groupby('member_casual')['ride_length'].agg(['mean', 'median', 'max', 'min']))

# See the average ride time by each day for members vs casual users
print("\nAverage ride length by member type and day of the week:\n",
all_trips_v2.groupby(['member_casual', 'day_of_week'])['ride_length'].mean())

# Order the days of the week for correct sorting for analysis
days_order = ["Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"]
all_trips_v2['day_of_week'] = pd.Categorical(all_trips_v2['day_of_week'],
categories=days_order, ordered=True)

# Now, run the aggregation again to see the sorted result
print("\nAverage ride length (sorted by day of week):\n", all_trips_v2.groupby(['member_casual',
'day_of_week'])['ride_length'].mean())

# Analyze ridership data by type and weekday
summary_stats = all_trips_v2.groupby(['member_casual', 'day_of_week']).agg(
    number_of_rides=('ride_id', 'count'),
    average_duration=('ride_length', 'mean')
).reset_index()

print("\nSummary of rides and duration by rider type and weekday:\n", summary_stats)

# You can further explore the data by examining specific relationships between values.
# For example:
# The number of rides by rider type and weekday
# The average ride duration by rider type and weekday

#=====
# EXPORT SUMMARY FILE FOR FURTHER ANALYSIS
#=====

```

```
# Create a .csv file that you will visualize elsewhere
counts = all_trips_v2.groupby(['member_casual',
'day_of_week'])['ride_length'].mean().reset_index()
counts.to_csv('avg_ride_length.csv', index=False)
```